Direct Publication for Offline Ballots

A Smaller Tally Path with Fewer Parties

João Henrique Menezes Silva Boia

Independent researcher

October 2026

Abstract

A tally is easier to dispute when several parties touch a ballot before anyone can check it. This study specifies a shorter path. The machine stays offline while the voter chooses. It prints a receipt that does not name the candidate, and only then does the voter confirm or test. A test opens that ballot and spoils it. A confirmed ballot stays encrypted. After the close, the same machine goes online and sends one signed batch straight to a public contract. No intermediary receives or rewrites the votes. The contract accepts the batch only when the registered machine signed it, and it accepts one batch from that machine. Three of five trustees then open the encrypted sum, not the individual ballots. A separate program checks a supplied receipt and recomputes the tally from the public record. The accompanying implementation exercises eight failures on this path and records a local run: six confirmed ballots, one spoiled ballot, and three timing samples for each measured operation. The design uses established cryptography. It does not claim a new primitive, a national deployment, or the absence of audits in elections that already publish their records.

1. Introduction

An election has to hide each choice and still let an outsider check the total. Those two requirements are not met by a screen that says the vote was counted, or by a file that some office forwards and then announces. Every extra handoff is another place where a ballot can be dropped, edited, or replaced by a total that nobody else can recompute.

The path studied here has four roles. The offline machine prepares the encrypted ballot and the receipt. A public contract records the machine's signed batch. Five trustees hold shares of the decryption key, and any three can open the sum. An independent verifier recomputes the result from the published evidence. The machine publishes its own batch. A poll worker may enable the connection after the close, but that person does not receive the votes and does not create a second batch. A fallback copy, if one is used, carries the same signed bytes.

This is a study of that shorter path, together with a local implementation that makes the checks executable. It is not a proposal to replace every existing election procedure, and it is not a claim that current elections have no audits. The concrete difference is who is allowed to handle the ballot bytes. The contract will not take a batch from an unregistered signer, a second batch from the same machine, or a payload whose signature no longer matches. The verifier will not accept a tally that disagrees with the opened sum.

The argument for fewer parties is operational. A path fails in more ways when a carrier, a central counter, and the machine can each produce a different story. Removing the carrier removes the step at which signed

bytes can be edited in transit. Keeping the opening threshold at three of five means that one trustee cannot announce a private decryption, and that the public result does not depend on all five being present. The cost of that choice is stated later: any three trustees can still decrypt individual ciphertexts, and the local demonstration runs on one chain process controlled by one operator.

2. The shorter path

2.1 Roles

Table 1 compares the path in this study with collection through an intermediary. The right-hand column is a pattern, not a description of one country's election. Brazil already votes offline, prints a polling-station bulletin, gives the voter an attendance acknowledgement, and publishes the digital vote record and other files for audit. The bulletin reports a station total. The acknowledgement records participation. The digital vote record is not a cryptographic receipt held by the voter. Those facts are the baseline. This study does not treat them as missing, and it does not treat a blockchain as the only possible public record.

Table 1: Who touches the ballot. The right-hand column is a comparison pattern, not one country's election.

Step	This study	Through an intermediary
Voting	Offline machine	Client, offline or online
Evidence	Receipt before the choice	Participation slip, or none
After close	Machine sends one signed batch	An office receives the file
Acceptance	Contract checks that signature	That office accepts the file
Opening	Three of five open the sum	One office announces the total
Check	A separate program recomputes	Observers read the publication

The conflict this design removes is the one in which the same organization operates the machines, carries the results, and is also the only party that computes the announcement. Here the machine can publish only what it signs. The contract does not open ballots. The trustees contribute shares for the sum. The verifier does not need the machine's program. A person who holds a receipt can ask whether that ballot is in the published batch.

What remains is also a conflict, and it should be named. The dealer who creates the key shares is trusted. The demonstration process holds all five shares. Any three trustees can decrypt a ciphertext, so the rule that the public interface opens only the sum is a rule of the software, not a cryptographic barrier against those three. A machine can still encrypt a choice the voter did not make; the test ballot is the check, and it catches only the ballots that are tested. One operator of the local chain can reset or censor that chain. Copied event data is not proof that independent validators agreed on one history.

2.2 Why fewer parties reduce the ways to fail

The implementation draws a hard boundary at each remaining role.

- The voting program cannot turn the network on. A separate enablement, modeled as a poll worker's action after the close, opens the simulated transport. An early attempt is rejected.
- The contract stores the signed payload. It checks the registered signer, the deadline, and the one-batch rule. It does not re-encode the ballots.

- A changed byte breaks the machine signature. The contract rejects that submission, and so does the verifier.
- A dropped ballot is visible when the voter supplies the receipt that was printed before confirmation. Without that receipt, omission need not leave a trace. The study does not add a voter roll, so it does not prove that every eligible person voted once.
- A false announcement fails when the verifier rebuilds the total from confirmed ballots and checked shares.

Fewer parties are not the same thing as fewer failures of every kind. The machine is still a single place that sees the choice in the clear. The chain operator is still a single place that can refuse the transaction. The gain is that those failures are no longer mixed with a quiet edit by someone standing between the machine and the record.

3. Protocol

The group is RFC 3526 group 14: a 2048-bit prime p, order $q = (p - 1)/2$, and generator $g = 2$ [7]. Secret exponents lie in $\mathbb{Z}_q$, and the public key is $y = g^x$. For a bit m and fresh randomness r, exponential ElGamal is

$$\mathrm{Enc}_y(m; r) = (a, b) = (g^r,\ y^r g^m) \pmod{p}. \tag{1}$$

A ballot is one ciphertext per candidate. Each ciphertext carries a Fiat–Shamir OR proof that it encrypts zero or one. The challenge binds a domain separator, the election identifier, the public key, the whole ordered ciphertext vector, and the candidate index, so a proof cannot be moved to another election, key, ballot, or position. A second proof shows that the candidate bits sum to one. With $A = \prod_i a_i$, $B = \prod_i b_i$, and witness $R = \sum_i r_i \bmod q$,

$$\log_g A = \log_y (B/g) = R \pmod{q}. \tag{2}$$

The transcript is canonical JSON, hashed with SHA-256, and reduced modulo q. Subgroup membership, scalar ranges, encodings, and the proof equations are checked before a ballot is accepted. The equality proof follows Chaum and Pedersen [5]. The sharing follows Shamir [10]. Neither construction is new here.

3.1 Receipt, then confirm or test

The machine encrypts the choice and signs the receipt before the voter confirms or tests. The receipt binds the election, the chain, the board, the machine account, and a SHA-256 commitment to the ordered ciphertext vector. Proof bytes are not part of that commitment. A 20-hex-character code is an 80-bit locator for lookup. It is not the binding value. Verification uses the full digest and the signature.

On a test, the machine reveals the choice and the randomness. The verifier recomputes the ciphertexts, checks the opening, and marks the ballot spoiled. That ballot is not counted, and the voter votes again. On confirmation, the randomness stays undisclosed and the ciphertext is marked cast. The receipt does not prove that a physical button was pressed, and it does not decide eligibility.

3.2 Direct publication

Local voting closes first. The voting program cannot enable transport before that close. After the close, the machine publishes one signed batch before the upload cutoff. The cutoff is not the same event as the local

close: machines do not have to finish closing before any machine may upload. When the batch deadline passes, the board closes and a snapshot is fixed. Trustee shares are accepted only after that snapshot and before a separate deadline.

The batch is signed by the same secp256k1 account that submits the transaction. The EIP-191 digest binds the chain, the board, the election, and the payload. The contract checks the signer, the deadline, and the one-batch rule, then emits the payload and its hash. The verifier checks the event and the signature. Nothing in this step decrypts a ballot.

3.3 Opening only the sum

The verifier multiplies the ciphertexts of cast ballots after the board has closed. Spoiled ballots stay out. For a product (A, B) and a trustee share s_i, trustee i posts $D_i = A^{s_i}$ and a Chaum–Pedersen proof that $\log_g VK_i = \log_A D_i$, where $VK_i = g^{s_i}$. Each proof binds the election, the aggregate digest, the trustee index, the candidate, the key, the ciphertext, and the commitments. For a set S of three trustee indices,

$$\lambda_i = \prod_{\substack{j \in S \\ j \neq i}} \frac{-j}{i-j} \pmod{q}, \qquad D = \prod_{i \in S} D_i^{\lambda_i}. \tag{3}$$

The plaintext group element is $B/D = g^T$. The verifier solves for T inside the declared tally bound. The result is the aggregate. It is not a cryptographic guarantee against three trustees who decide to decrypt some other ciphertext.

4. What the checks catch

The eight commands below are executable detections. They show that the corresponding check fires on the implementation. They are not a proof that every attack outside this list fails.

- **Substituted choice.** A test opening that disagrees with the ciphertext is rejected (`attack-machine`). Untested ballots are not covered by that one test.
- **Edited transport.** Changing the signed bytes invalidates the signature, and the contract rejects the submission (`attack-transport`).
- **Dropped ballot.** A retained receipt that is absent from the signed batch fails the check (`attack-drop-vote`).
- **False tally.** An announced total that disagrees with the recomputation is rejected (`attack-tally`). The verifier uses the public evidence, not a private list of expected choices.
- **Two selections.** A ballot with two selected candidates fails the exactly-one proof (`attack-double-vote`). The proof does not stop someone from submitting several valid ballots.
- **Second batch.** The contract rejects another batch from a registered machine while the upload window is still open (`attack-duplicate`).
- **Unknown machine.** The contract rejects a signer that was never registered (`attack-rogue-machine`). Registration itself is an external anchor.

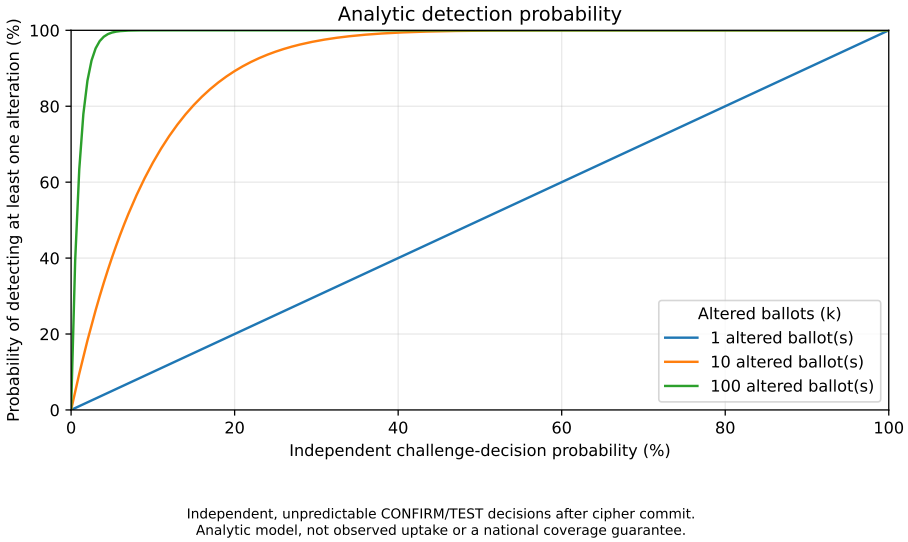


Figure 1: Analytic probability $1-(1-p)^k$ of detecting at least one altered ballot when tests are independent and chosen after the receipt exists. The curves are the equation, not observed detection rates.

- **Early network.** The voting program cannot enable the simulated transport before the close (`attack-early-network`). This is a property of the software state, not of a physical switch.

If each ballot is tested independently with probability p after the receipt exists, and k ballots were altered, the chance of catching at least one alteration is $1-(1-p)^k$. Figure 1 plots that expression. It is not a measured detection rate. It assumes the decision is unpredictable and independent of what the machine knows. It does not model coercion, correlated tests, or malware that adapts only to ballots it expects to remain untested.

5. Measurements

The implementation has three parts: an offline Python machine, a Solidity contract, and a separate Python verifier. The reported run uses one local Anvil process. It is evidence about this implementation, not a fee schedule for a public chain and not a benchmark campaign on other hardware. Tool installation and the first container build are excluded. The recorded environment is arm, Python 3.12.11, Anvil 1.8.5, and Darwin 27.2.0.

The run contains six cast ballots, one spoiled ballot, three candidates, two machines, two batches, and three partial decryptions. Each pure operation has three samples. Three fresh process-and-node runs of the full demonstration took 56.590681917 s, 56.174527333 s, and 56.164920292 s. The first of these is the first recorded run, not a run on flushed caches. Three isolated verifier processes took 17.819719167 s, 17.830205542 s, and 17.814804209 s.

Bit-proof time includes ciphertext setup and opening checks. Full-ballot time includes those proofs and the sum proof; it does not isolate the sum proof. Figure 2 shows the same three-sample medians and ranges. Three samples support a median and a range. They do not support a confidence interval or a prediction for another processor.

The two payloads together contain 76,633 bytes, including headers and the spoiled ballot. Proof bytes in

Table 2: Pure-operation time from three samples. The range is the minimum and the maximum.

Operation	Median (s)	Range (s)
Three-candidate encryption	0.151508750	0.151100–0.151819
Bit proofs, including opening checks	1.088121958	1.085648–1.126727
Full ballot, including the sum proof	1.137149792	1.134560–1.209563
Public verification of one ballot	0.827160750	0.823534–0.833354

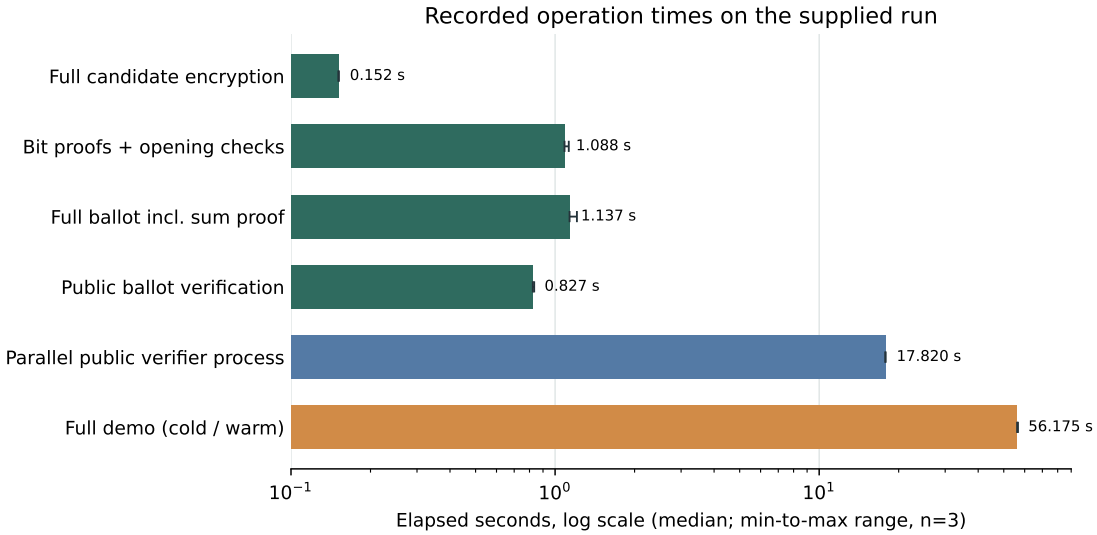


Figure 2: Medians and minimum-to-maximum ranges from the supplied measurement record, three samples each. The axis is logarithmic elapsed time.

those batches total 50,974. One canonical cast record in the measurement file is 10,622 bytes. Each of the three partial decryptions used 219,600 gas.

5.1 What the same numbers say at national scale

Divide 150,000,000 voters by 500,000 machines and the quotient is 300 voters per machine. Multiplying the measured 10,622-byte cast record by 150,000,000, and ignoring headers, spoiled ballots, and replication, gives 1.5933 decimal terabytes, or 3.1866 megabytes on each machine. That product is not a delivery estimate. The published payloads above are larger per ballot once headers and a spoiled ballot are included.

Repeating the gas of one three-ballot batch as if it were a fixed cost gives $100 \times 1{,}314{,}280 = 131{,}428{,}000$ gas for 300 cast ballots, and 6.5714×10^{13} gas for 150 million voters. This repeats one small measurement. It is not a large-batch experiment, a throughput result, or a statement that the implementation can run at that size. The code caps the demonstration at 16 machines, a tally of 16,000, and a 16 MiB payload. No partitioning is implemented.

Multiplying the median public verification time by 150 million ballots gives 124,074,112.5 sequential core-seconds, about 3.93 core-years. That figure prices one checked ballot and nothing else: not setup, not the aggregate, not disk, and not a slower proof. It does not say that a home computer can check a national election quickly.

The useful reading of these products is narrow. The short path is small in the number of parties. It is not small in bytes or gas if every proof is published on a chain at national volume. A deployment at that volume

Table 3: Payload size and gas for the two published batches on the local chain. Gas is an execution unit, not a currency fee.

Batch	Payload (bytes)	Gas	Cast ballots
First machine, mixed	44,416	1,802,170	3
Second machine, cast only	32,217	1,314,280	3

would have to change the packaging, the chain, or both. This study does not make that change.

6. Related work

Helios showed that a remote voter can audit an encrypted ballot, and it uses a mixnet. Its setting does not remove coercion, and a web client is a different trust boundary from a polling-place machine [1]. Benaloh's verifiable elections are the reference for a voter checking an encrypted choice [2]. ElectionGuard specifies a full system, with a different group and encryption construction; nothing here claims compatibility or equal assurance [3]. Scantegrity II puts end-to-end checks on optical-scan paper [4]. vVote describes a polling-place system whose operational assumptions are not the ones implemented here [6].

Those systems are larger on purpose. They include credentials, paper processes, or mixnets that this study leaves out. The omission is the method: fewer roles, a direct signed publication, and a threshold that opens the sum. It is not an argument that a smaller codebase is safer in an election that needs eligibility, accessibility, and a legal dispute process.

Park, Specter, Narula, and Rivest argue that moving an election onto a network or a chain can add malware, coercion, censorship, and a weak dispute story rather than remove them [9]. Direct publication answers only the intermediary. It does not answer a compromised machine, a coerced voter, or a chain that refuses the transaction. Risk-limiting audits need voter-verifiable paper records [8]. A ciphertext receipt does not replace those records.

7. Limits

The dealer is a single trusted setup. There is no distributed key generation. Three trustees can read individual ballots. The demonstration holds every share. The implementation is ordinary Python, so it does not hide secrets from a program that logs them. The simulated seal, power cut, reinstall, and hash comparison do not prove that a particular device ran a particular binary.

There is no voter list, no turnout reconciliation, and no one-person-one-vote check. The contract does not verify ballot proofs, trustee proofs, or the tally arithmetic. A bad proof stops the verifier. A wrong first pin of the aggregate can stop honest shares from matching. The local chain can be reset. None of the eight commands measures inclusion against an adversary who simply never publishes, or against a voter who loses the receipt.

8. Conclusion

The path from an offline ballot to a public tally can be short. The machine signs one batch and sends it. The contract accepts that signature and no other party's repackaging. Three of five trustees open the sum. A separate program checks the receipt and recomputes the total. Each of those restrictions removes a place where

an intermediary could have changed the story. The local run shows the checks firing, and the measurements show what the current packaging costs. They do not show a national election, a new cryptosystem, or a tally that remains correct when the machine, three trustees, or the chain operator abandon the protocol.

Availability

The implementation, the eight attack commands, and the measurement record accompany this manuscript. The code is under the Apache License 2.0. This PDF is the preprint. It is not a journal article, and no institutional affiliation is claimed. An arXiv identifier can be added when the preprint is posted.

References

[1] Ben Adida. Helios: Web-based open-audit voting. In *17th USENIX Security Symposium*, 2008.

[2] Josh Benaloh. Simple verifiable elections. In *Proceedings of the USENIX/Accurate Electronic Voting Technology Workshop*, 2006.

[3] Josh Benaloh, Michael Naehrig, and Olivier Pereira. Electionguard design specification. Technical specification Version 2.1.0, Microsoft Research, August 2024.

[4] David Chaum, Richard Carback, Jeremy Clark, Aleksander Essex, Stefan Popoveniuc, Ronald L. Rivest, Peter Y. A. Ryan, Emily Shen, and Alan T. Sherman. Scantegrity ii: End-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In *Proceedings of the USENIX/ACCURATE Electronic Voting Technology Workshop*, 2008.

[5] David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Advances in Cryptology, CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*. Springer, 1993.

[6] Chris Culnane, Peter Y. A. Ryan, Steve Schneider, and Vanessa Teague. vvote: a verifiable voting system. arXiv:1404.6822v4, 2015.

[7] Tero Kivinen and M. J. Kojo. More modular exponential (modp) diffie-hellman groups for internet key exchange (ike). RFC 3526, 2003.

[8] Mark Lindeman and Philip B. Stark. A gentle introduction to risk-limiting audits. *IEEE Security and Privacy*, 10(5), 2012.

[9] Sunoo Park, Michael Specter, Neha Narula, and Ronald L. Rivest. Going from bad to worse: from internet voting to blockchain voting. *Journal of Cybersecurity*, 7(1), 2021.

[10] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11), 1979.